

Conversational AI

Session 1: Foundations of Conversational and Agentic AI

BITS Pilani

Dr Bharathi R
Adjunct Professor, CSIS WILP BITS PILANI



innovate achieve lead

BITS Pilani

AIMLCZG521, Conversational AI

Session 1 | Module 1

Disclaimer & Acknowledgement

innovate achieve lead

DISCLAIMER

These instructional materials have been developed by the instructor for educational purposes. All content is intended solely to support the learning objectives of this course. While every effort has been made to ensure accuracy and completeness, learners are encouraged to consult original source publications for authoritative reference.

ACKNOWLEDGEMENT

These instructional materials have been developed by the instructor with due acknowledgement of the original publication authors and the broader academic community for their openly accessible resources, which have been adapted and modified to support the learning objectives of this course.

Course Architecture: 4 Modules, 16 Lectures

The Paradigm Shift: Conversational AI has moved from rule-based systems to autonomous agents. This course covers current practice and emerging trends.

01 Foundations L1–L3

- Foundations of Conversational & Agentic AI; LLM Foundations
- Prompting Techniques, Structured Outputs & Response Control
- System Architecture for Conversational AI

02 Core Building Blocks L4–L8

- Tool Integration, Function calling & ReAct framework
- Agent memory: short-term, long-term, hybrid
- Embeddings, Vector Databases
- RAG pipelines: chunking, re-ranking, contextual retrieval

03 Autonomous Agents L9–L11

- Agent planning & multi-agent systems
- State management, hierarchical architectures
- RAG & agent evaluation (LLM-as-Judge)
- Cost optimisation & model routing

04 Production Ecosystem L12–L16

- Security: prompt injection, PII, red-teaming
- MCP deep dive: client-server, resources, tools
- A2A protocol & agent interoperability
- Ethics, governance & bias mitigation

What You Will Build: Learning Outcomes

LO1 **End-to-End Conversational AI Systems**

Design agents that use vector databases for retrieval, function calling for tools, and graph-based workflows for complex reasoning.

LO2 **Cost-Effective Production Solutions**

Apply prompt caching, model routing and efficient retrieval to cut token costs by up to 90%.

LO3 **Secure & Observable Agents**

Defend against prompt injection, protect PII, and monitor performance, cost and errors.

LO4 **Ethical & Governed AI Agents**

Mitigate bias, keep humans in the loop for critical decisions, and comply with AI regulations (e.g., EU AI Act).

Evaluation Scheme & Important Rules

10%	**EC-1: Quizzes (×2)** Open book	
20%	**EC-1: Assignments (×2)** Open book	
30%	**EC-2: Mid-Term Exam** Closed book (L1–L8)	As per calendar
40%	**EC-3: End-Semester Exam** Open book (L1–L16)	As per calendar

⚠ **Important: No deadline extensions · No make-ups for quizzes or assignments · Plagiarism = zero marks + disciplinary action**

Prerequisites

What You Need Coming In



Python: Functions, classes, basic libraries



ML Basics: Training, inference, evaluation



Deep Learning: Neural networks, transformers



API Basics: REST calls, JSON data handling



Basic Statistics: Precision, recall, probability

Tips

1. **Code every lecture:**
   Run the demo code and change one thing.
2. **Use the references:**
   ReAct, DPR and DPO papers: read the abstract and results.
3. **Get an API key / use open source models**
   OpenAI, Anthropic or Google Gemini; low-cost tiers cover the demos.
4. **Start assignments early:**
   No extensions;
5. **Ask on Canvas:**
   Peer discussion is encouraged.

Outline

1. Introduction to Conversational AI
2. Evolution from Chatbots to Autonomous Agents
3. Building Blocks: Components, LLMs and Their Limits
4. Agent Lifecycle: Reasoning, Planning and Action Loops
5. Protocols and Production Concerns
6. NLP Foundations: Tokenization and Context Windows
7. Open Problems and Key Takeaways

References: T1 – Jurafsky & Martin, Speech and Language Processing, Ch. 1; R1 – Wang et al. (2024), LLM-based Autonomous Agents

1. Introduction to Conversational AI

What Is Conversational AI?

Definition: An AI system that converses in natural language to understand intent, retain context, retrieve knowledge, and respond or take action.

Core Capabilities

Natural Language Understanding (NLU)
Understanding user intent and entities

Dialogue Management
Managing conversation flow and context

Natural Language Generation (NLG)
Generating human-like responses

Context Awareness
Remembering conversation history

Multi-turn Interaction
Handling complex dialogues

Personalization
Adapting to user preferences



Understand

Interpret natural language intent, entities, sentiment



Reason

Plan, chain thoughts, decompose multi-step problems



Act

Call APIs, write code, retrieve docs, orchestrate agents

Source: T1 – Jurafsky & Martin, Ch. 1

Examples in Daily Life



Voice Assistants: Alexa, Siri, Google Assistant

Wake word → ASR → NLU → Action → TTS



Customer Support: Banking chatbots, e-commerce help

Handle tier-1 queries 24/7; escalate complex cases to humans.



Virtual Agents: Healthcare triage, HR assistants

Triage symptoms and suggest care pathways (safety-critical).

Market Trends & Industry Impact

$41.39B

Conv-AI Market by 2030

Grand View Research (2025)

100M

ChatGPT Users in 2 Months

Fastest-growing consumer app (UBS, 2023)

80%

Fortune 500 Using AI Agents

Microsoft (Nov 2025)

Industry Applications & Impact

Banking & Finance

~70% of tier-1 queries handled by AI; up to 60% cost reduction

Sources: CoinLaw; AutomationEdge (2025)

Healthcare

AI triage cuts ER wait times; enables 24/7 remote monitoring

Source: McKinsey (2025)

Enterprise Software

Agents browse, code, draft contracts and operate tools

Sources: Gartner (2025); DigitalDefynd

E-Commerce

15–26% conversion lift from AI-personalised recommendations

Sources: Barilliance; MindStudio

2. Evolution from Chatbots to Autonomous Agents

The Evolution Journey (1960s–2026)

Era	Technology	Capabilities	Limitations
1960s–1990s	Rule-Based (ELIZA, ALICE)	Pattern matching, keyword detection, simple scripted Q&A	No learning, rigid, no context
2000–2010	Statistical ML (SVMs, CRFs, HMMs)	Intent classification, entity extraction	Limited context, hand-crafted features
2010–2017	Deep Learning (RNNs, LSTMs, Seq2Seq, Attention)	End-to-end learning, better context understanding	Data hungry, task-specific training
2017–2020	Transformers (BERT, GPT-2, T5)	Transfer learning, contextual embeddings	Still requires task-specific fine-tuning
2020–2023	LLMs & Generative AI (GPT-3, ChatGPT, Claude, PaLM)	Few-shot learning, general-purpose, fluent generation	Hallucinations, no real-time data, no actions
2023–2025	Agentic AI (LLMs + Tools + Memory + Planning)	Execute actions, multi-step reasoning, autonomous workflows	Complex orchestration, scaling challenges, cost optimization
2025–2026	On-Device & Multi-Modal (SLMs, native multimodality)	Real-time voice/video, privacy-first local processing	Hardware constraints, fragmented ecosystems

Key driver: The Transformer (2017), affordable GPU compute and internet-scale data together made LLMs possible.

& Martin, Ch. 1; Vaswani et al. (2017)

The Evolution of Conversational AI

From scripted responses to goal-directed autonomy: the levels differ in how much initiative and control the system has.

LEVEL 1

Rule-Based Chatbots

Rule-driven & reactive

- Keyword matching
- Scripts and decision trees
- Limited conversational context

ANSWERS QUESTIONS

LEVEL 2

LLM-Powered Assistants

Context-aware, tool-enabled & memory-enabled

- LLM-based understanding and generation
- Tool calling / API integration
- Conversation history, memory and RAG

ASSISTS WITH TASKS

LEVEL 3

Autonomous AI Agents

Goal-directed & adaptive

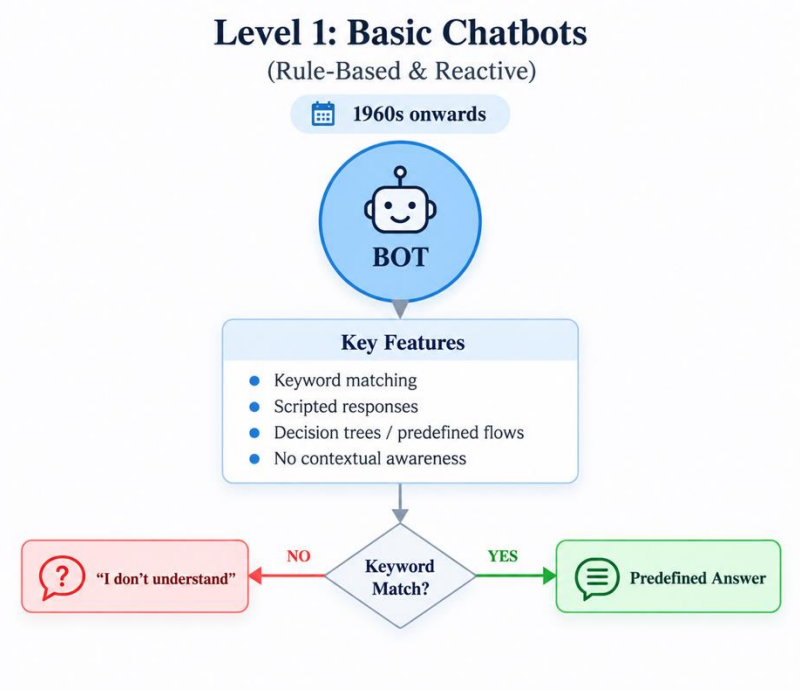
- Plans and decomposes goals
- Acts, observes and revises
- Coordinates multi-step workflows

MANAGES OUTCOMES

l. (2024)

Level 1: Rule-Based Chatbots

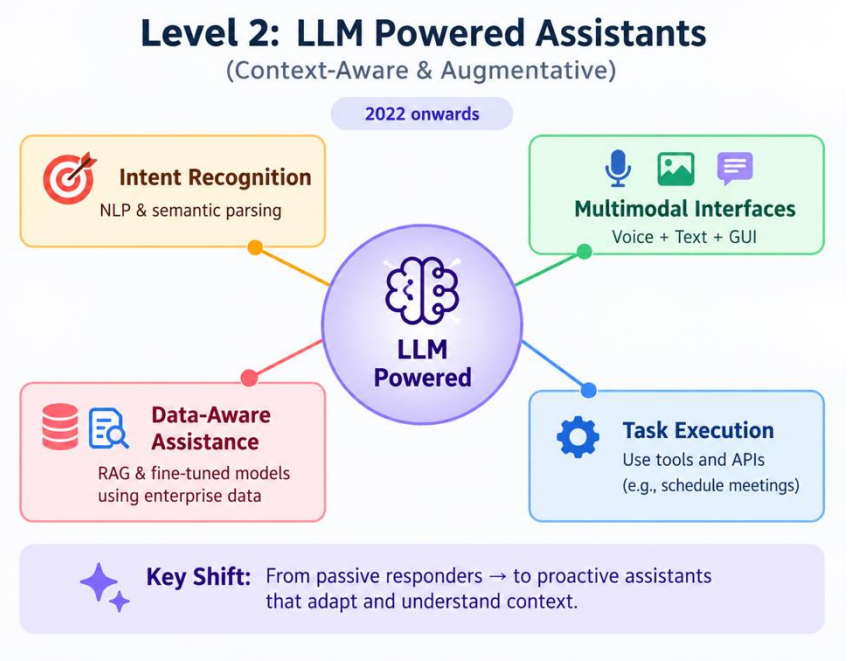
- **Functionality:** Keyword matching, scripted replies and decision trees.
- **Use cases:** FAQs, support widgets, static knowledge-base queries.
- **Limitations:** No context; fails when input leaves predefined paths.
- **Examples: ELIZA (1966), first-generation customer-service bots.**



Sources: T1 – Jurafsky & Martin, Ch. 1; Weizenbaum (1966)

Level 2: LLM-Powered Assistants

- **Intent recognition:** NLU identifies user goals (e.g., ChatGPT, Google Assistant).
- **Multimodal input:** Text, voice and images (e.g., **Gemini, ChatGPT**).
- **Data-aware answers:** RAG over enterprise data (e.g., Amazon Q Business, IBM watsonx Assistant, **Notebook LLM**).
- **Task assistance:** Tools and APIs to schedule, search and draft (e.g., Gemini, Amazon Q Business).
- **Key shift:** From reactive responders to context-aware assistants.

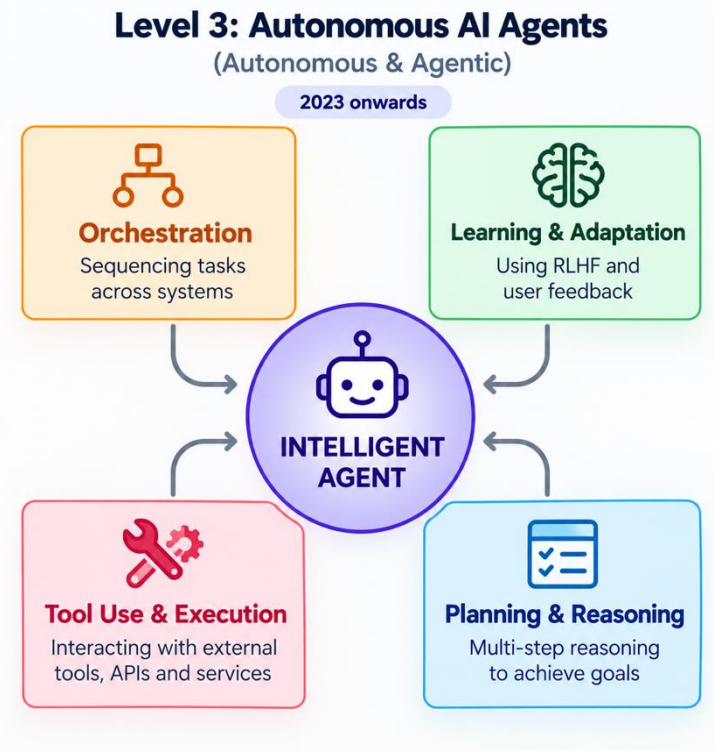


Note: Examples are representative; capabilities vary by version and configuration.

Sources: R1 – Wang et al. (2024); Lewis et al. (2020)

Level 3: Autonomous AI Agents

Planning & tool use: Breaks goals into subtasks, calls tools, checks results and revises the plan.
Orchestration: Coordinates tasks across systems (e.g., a procurement agent using ERP, vendor APIs and legal databases).
Learning & adaptation: Improves through runtime feedback and memory; RLHF applies at model-training time.
Enterprise integration: Acts inside workflows within granted permissions (e.g., rerouting orders during supply-chain disruptions).
Paradigm shift: From assistants that help with tasks to agents that pursue goals.
Examples: AutoGPT, BabyAGI, Claude Code, Amazon Bedrock Agents, AgnetForce.



al. (2024); Yao et al. (2023)

Level 2 vs Level 3

Tools and memory exist at both levels; Level 3 adds an iterative plan → act → evaluate loop.

LEVEL 2 | LLM + TOOLS + MEMORY

- User request
- ↓
- LLM interprets context
- ↓
- Retrieve information / call a tool
- ↓
- Return result for user review

Often request-driven; user guides the task

LEVEL 3 | AI AGENT

- User specifies a goal
- ↓
- Plan and select an action
- ↓
- Act using tools
- ↓
- Observe and evaluate result

Repeat / revise until done or approval is needed

Sources: R1 – Wang et al. (2024); Yao et al. (2023)

Use Case: Customer Support Evolution

Scenario: Banking Customer Support

Approach	Capabilities	User Experience
Rule-Based (2000s)	• FAQ matching • Keyword detection • Fixed responses	User: "I lost my card" Bot: "For lost card, press 1. For stolen, press 2" Rigid, frustrating
Intent-Based (2015)	• Intent classification • Entity extraction • Dialogue flow	User: "I lost my card" Bot: "I'll help you block your card. Which card - Credit or Debit?" Better, but limited
LLM-based (2023)	• Natural conversation • Context understanding • Fluent responses	User: "I think I left my card at the restaurant" Bot: "I understand. Let me help you block it temporarily while you check..." Natural, but no action
Agentic AI (2025)	• Everything above + • Access banking system • Execute transactions • Multi-step actions	User: "I lost my card at City Mall" Bot: "I've immediately blocked your card ending in 1234. I see recent transactions at City Mall - last one was $45.20 at Starbucks 2 hours ago. Should I order a replacement card to your home address?" Proactive, action-oriented

Illustrative impact: Resolution time drops from ~15 min (human agent) to ~2 min (agentic AI), with higher customer satisfaction.

3. Building Blocks: Components, LLMs and Their Limits

Conversational AI Components (1/3)

1. Natural Language Understanding

Role: Understand what user wants

- Intent classification
- Entity extraction
- Sentiment analysis
- Context understanding

Modern approach: LLM-based, single pass

2. Dialogue Management

Role: Manage conversation flow

- State tracking
- Context maintenance
- Turn-taking
- Error handling

Modern approach: LLM reasoning + Memory systems

Conversational AI Components (2/3)

3. Knowledge Access

Role: Retrieve relevant information

- Vector databases
- Semantic search
- RAG (Retrieval-Augmented Generation)
- Real-time data access

Modern approach: Embeddings + Hybrid retrieval

4. Action Execution

Role: Take actions on behalf of user

- Tool/function calling
- API integrations
- Database operations
- External service invocation

Modern approach: Agentic tool use

Sources: Lewis et al. (2020); R1 – Wang et al. (2024)

Conversational AI Components (3/3)

5. Response Generation

Role: Generate natural responses

- Contextual generation
- Personality/tone
- Multi-modal output
- Structured responses

Modern approach: LLM generation with control

6. Memory Systems

Role: Remember user context

- Short-term (conversation)
- Long-term (user profile)
- Episodic memory
- Semantic memory

Modern approach: Vector + SQL hybrid

Source: R1 – Wang et al. (2024)

Transformers & LLMs: The Brain

Transformer (2017) → Pre-training on Massive Data → LLM (GPT / Claude / Llama) → Fine-tuning / Instruction Tuning → Tools + Memory + Planning → Conversational Agent

Technical Capabilities	*enables*	Impact on Conversations
Self-Attention Relate every word to all others in the input	→	**Ambiguity Handling** Resolve unclear or underspecified requests
Contextual Understanding Word meaning depends on surrounding context	→	**Intent Understanding** Grasp user goals from conversational context
Long-Range Dependencies Connect ideas across very long passages	→	**Multi-turn Coherence** Remember and track earlier parts of a conversation
Parallel Processing Faster training than RNNs, enabling massive scale	→	**Natural Responses** Fluent, human-like replies (indirect, via scale)

Key insight: Each internal capability of the Transformer shows up as a visible conversational behavior.

LLM Capabilities and Limitations

✅ What LLMs Do Well

- **Natural conversation:** Human-like dialogue
- **Intent understanding:** Grasp user goals
- **Context retention:** Multi-turn conversations
- **Generation:** Fluent, contextual replies
- **Summarization:** Condense information
- **Entity extraction:** Identify key information
- **Sentiment analysis:** Detect user emotions

✗ LLM Limitations

- **No real-time data:** Training cutoff date
- **Hallucinations:** Plausible but false output
- **Can't act:** No access to external systems
- **No memory:** Forget after context window
- **Calculation errors:** Struggle with math
- **Inconsistency:** Varying answers to one question
- **No verification:** Can't check facts

Source: Bommasani et al. (2021)

Why We Need Agentic Architecture

Solution: Augment LLMs with external capabilities

LLM Limitation	Agentic Solution	Example
No real-time data	Tool calling (APIs, search)	Check current flight prices
Hallucinations	RAG (Retrieval-Augmented Generation)	Ground answers in company docs
Can't take actions	Function calling	Book appointment, send email
No memory	External memory systems	Remember user preferences
Calculation errors	Calculator tool, code execution	Precise math computations

Sources: Lewis et al. (2020); Yao et al. (2023)

4. Agent Lifecycle: Reasoning, Planning and Action Loops

Agent Lifecycle: Request → Response

Seven stages; Reasoning ↔ Tool Invocation repeats as a loop until the goal is met (ReAct).

1 REQUEST

Receive input · Validate & sanitize · Extract entities

2 ROUTING

Classify intent · Select tools · Route to sub-agent

3 REASONING

Break goal into steps · Find information gaps · Plan tool calls

4 TOOL INVOCATION

Call APIs · Query databases · Execute code

5 MEMORY

Store history · Update preferences · Maintain session state

6 SAFETY

Block toxic content · Check facts · Redact PII

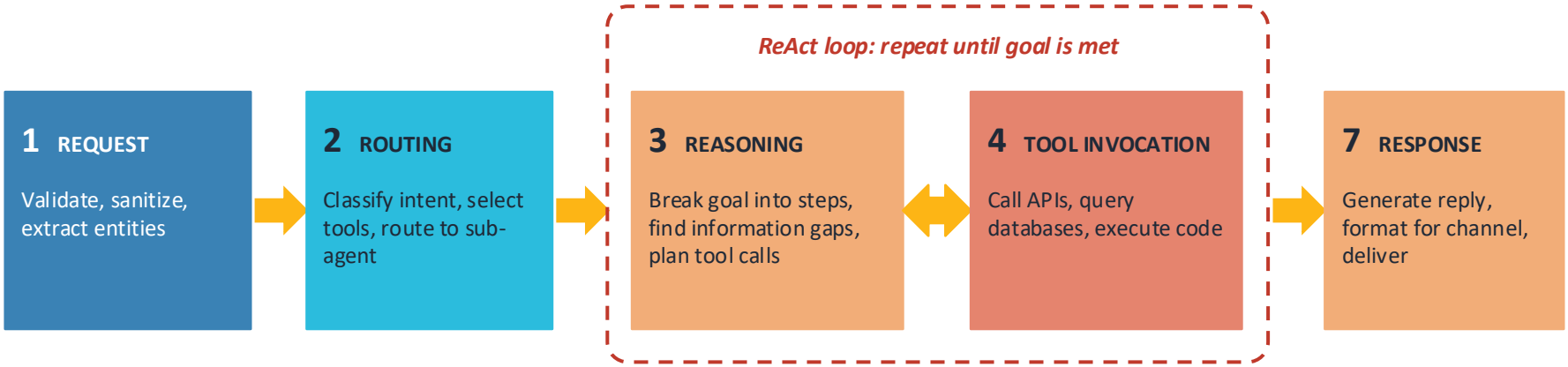
7 RESPONSE

Generate reply · Format for channel (text/voice/UI) · Deliver

Sources: R1 – Wang et al. (2024); Yao et al. (2023)

Agent Lifecycle: Request → Response

Seven stages; Reasoning ↔ Tool Invocation repeats as a loop until the goal is met (ReAct).



5 MEMORY Store history · Update preferences · Maintain session state

Cross-cutting: read and written at every stage, not only after the response

6 SAFETY Block toxic content · Check facts · Redact PII

Cross-cutting: guards the input on the way in and the output before delivery

Note: only stages 3 and 4 loop; a simple question can skip tools and go straight to Response.

Sources: R1 – Wang et al. (2024); Yao et al. (2023)

Lifecycle Example: Banking Agent

User: "What's my account balance and recent transactions?"

1. Request

```
Intent: account_inquiry
Entities: {type: [balance, transactions]}
Validate: user authenticated
```

2. Routing

```
Route to: Banking Agent
Tools: get_balance, get_transactions
Permission check: user has access
```

3. Reasoning (plan)

1. Fetch account balance
2. Fetch recent transactions (last 5)
3. Format response with both

4. Tool Invocation

```
Call: get_balance(user_id) → $5,432.10
Call: get_transactions(user_id, limit=5)
Returns: balance + 5 transaction records
```

5. Memory

Store: user asked about balance at 2:30 PM

Update: user prefers transaction details

Context: keep for follow-up questions

6. Safety (gate before delivery)

Validate: data belongs to the correct user

Check: no PII exposure in logs

Verify: response has only authorized info

7. Response

"Your current balance is $5,432.10. Recent transactions:

1. Starbucks – $4.50 (Today)
2. Amazon – $89.99 (Yesterday)
3. Shell Gas – $45.00 (Yesterday)
4. Grocery Store – $123.45 (2 days ago)
5. Netflix – $15.99 (3 days ago)

Would you like details on any of these?"

Illustrative example; single pass, so the ReAct loop is not needed.

Banking Agent Example: The ReAct Loop

User: "Why was I charged twice at Amazon?"

1. Request

```
Intent: billing_inquiry (user authenticated)
Entities: {merchant: Amazon, issue: duplicate}
```

2. Routing

```
Route to: Banking Agent (access checked)
Tools: get_transactions, get_transaction_details
```

ReAct loop: Reasoning ⇄ Tool Invocation, repeated until the goal is met

3. Reasoning (thought)	4. Tool Invocation (action → result)
Round 1: Need the recent Amazon charges first.	`get_transactions(user_id, merchant="Amazon", days=7)` → **2 charges of $89.99 on Oct 21**
Round 2: Same amount twice. Duplicate or two orders? Compare details.	`get_transaction_details(txn_1, txn_2)` → **same order; txn_2 is a pending authorization hold**
Round 3: Evidence is enough. Explain the cause and offer a next step.	`No tool call needed` → **exit loop; goal met**

5. Memory

Store: user asked about a duplicate Amazon charge
Context: keep both transaction IDs for follow-ups

6. Safety (before delivery)

Verify: both charges belong to this user
Check: no card numbers in reply or logs
Confirm: no dispute filed without consent

7. Response

"Both $89.99 charges come from one Amazon order. The second is a pending hold that should drop off in a few days. Want me to raise a dispute if it posts twice?"

Illustrative example; tool names and results are simulated.

Exercise: Map the Agent Lifecycle

Scenario:

"Find me a good Italian restaurant near my office that's open tonight and make a reservation for 2 at 7 PM"

Stage	What Needs to Happen?
1. Request	What entities need to be extracted from user input?
2. Routing	Which tools or agents are needed for this task?
3. Reasoning	What's the step-by-step plan to accomplish this?
4. Tool Invocation	What specific API calls need to be made?
5. Memory	What information should be stored for future use?
6. Safety	What validations are required before taking action?
7. Response	How should the agent communicate with the user?

Answer Key: Map the Agent Lifecycle

Scenario:

"Find me a good Italian restaurant near my office that's open tonight and make a reservation for 2 at 7 PM"

Stage	Model answer
1. Request	Intent: find and book a restaurant. Entities: cuisine = Italian, location = near office, date = tonight, time = 7 PM, party = 2, quality = "good". Office address must come from the user's profile or a question.
2. Routing	Places search tool + reservation (booking) tool: one agent with both, or separate search and booking sub-agents.
3. Reasoning	Resolve office location → search open, well-rated Italian places → check 7 PM for 2 → if full, try nearby times or the next option → confirm with user → book.
4. Tool Invocation	search_places(cuisine, location, open_now) · get_details(place_id) · check_availability(id, party=2, 19:00) · create_reservation(...) · add_to_calendar(...) (optional)
5. Memory	Store office location, cuisine preference, and booking details with confirmation number, so "move it to 7:30" or "cancel it" works.
6. Safety	Get user confirmation before booking (real-world action, possible deposit). Report only availability the tool confirmed. Share contact or payment details only with consent.
7. Response	Restaurant, address, time, party size, confirmation number, cancellation policy; offer directions or a calendar entry. If booking fails, offer alternatives.

If 7 PM is full, Reasoning ⇄ Tool Invocation repeats.

Hands-On: A Simple Conversational Agent

Demonstration: Weather Information Agent

Native OpenAI API in Python

Implementation Stages

1. **Baseline:** LLM without tools (observe its limits)
2. **Tool Definition:** Define weather function schema
3. **Tool Selection:** LLM decides when to use tool
4. **Execution:** Call actual weather API
5. **Response Generation:** LLM creates natural answer

Example Interaction

User: "What's the weather like in Mumbai?"

Agent:

1. Extracts location: "Mumbai"
2. Calls weather API
3. Receives: 32°C, Humid, Partly Cloudy
4. Responds: "It's currently 32°C in Mumbai with partly cloudy skies and high humidity."

Source: R1 – Wang et al. (2024)

5. Protocols and Production Concerns

Why Agent Protocols Matter

Agents need standard ways to connect to tools, data, other agents and user interfaces.

Without Standards

- Every vendor has custom API
- Agents can't interoperate
- Vendor lock-in
- Duplicated integration effort

With Standards

- Plug-and-play integrations
- Multi-agent collaboration
- Vendor flexibility
- Ecosystem growth

Protocol Landscape

MCP (Model Context Protocol) — By Anthropic (2024)

Standardize how LLMs connect to data sources and tools

Agent accessing DBs, files, APIs consistently

A2A Protocol (Agent-to-Agent) — Google (2025)

Standardized inter-agent communication

Cross-vendor multi-agent collaboration

OpenAI Responses API — OpenAI

Built-in tools: web search, file search, code interpreter

Successor to the Assistants API (2025)

Custom REST / GraphQL APIs — Universal

Traditional service integration

Enterprise systems, legacy integrations

LangGraph (Framework) — LangChain

Graph-based agent workflows and state management

Complex multi-step reasoning, agent coordination

ANP (Agent Network Protocol) — Emerging

Open standard for peer-to-peer agent discovery and communication

Decentralized agent ecosystems, cross-platform interoperability

Note: The landscape is evolving fast. MCP and A2A are gaining adoption, but many production systems still use custom APIs. Covered in Module 4.

Sources: Anthropic (2024), MCP; Google (2025), A2A Protocol

Production Concerns for Conversational AI

Working in development is not enough: agents must work reliably at scale.

Observability – what to track?

- Track conversation flows
- Monitor tool invocations
- Token usage per conversation
- Response latencies & error traces

Tools: LangSmith, Arize Phoenix, OpenTelemetry

Cost Management – Optimization Strategies

- Prompt caching (up to 90% lower input cost)
- Model routing (smaller models when appropriate)
- Token budgets per user/session
- Efficient retrieval (reduce context size)

Example: Customer support ≈ $0.02–0.10 per conversation

Latency Budgets – Target latencies

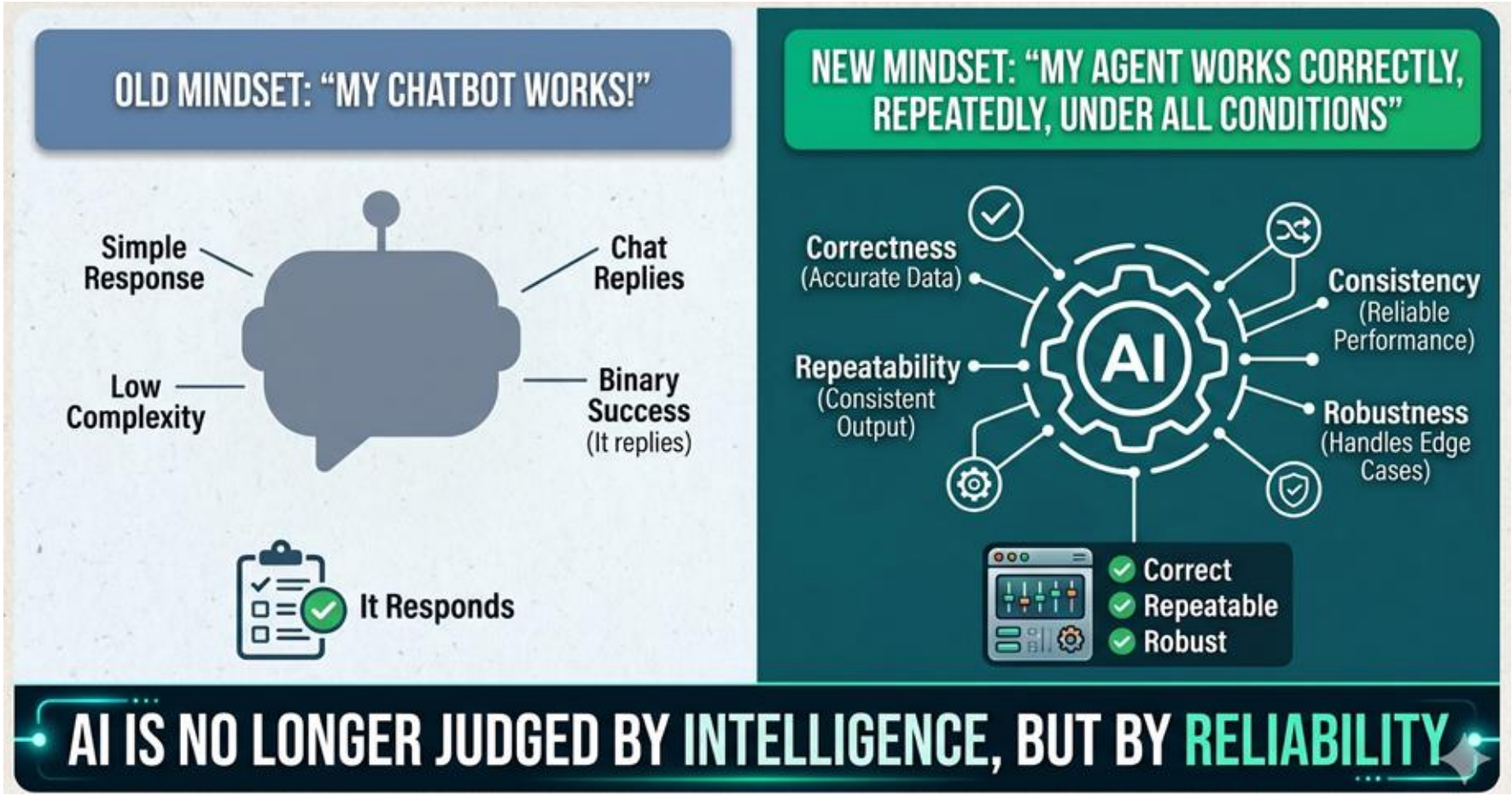
- Chat responses: < 2 seconds
- Tool execution: < 5 seconds
- Complex reasoning: < 30 seconds
- Set user expectations with progress indicators

Safety & Security – Defense layers

- Input validation (prompt injection defense)
- PII detection and redaction
- Output filtering (toxicity, hallucinations)
- Human-in-the-loop for critical actions

Principle: Never rely on a single safety layer

From Demo to Reliable Production



6. NLP Foundations: Tokenization and Context Windows

Tokenization: The Foundation

What is Tokenization?

Breaking text into subword units (tokens) that LLMs process.
Algorithms: BPE (GPT), SentencePiece (multilingual), WordPiece (BERT)

Text	Tokens	Count
"Hello World"	Hello · World	2 tokens
"artificial intelligence"	artificial · intelligence	2 tokens
"GPT-4"	G · PT · - · 4	4 tokens
"Book a flight to NYC"	Book · a · flight · to · NYC	5 tokens

Why It Matters for Conversational AI

Cost:
API pricing is per token → impacts conversation economics

Context Window:
Limits conversation length (200K tokens ≈ 150K words)

Latency:
More tokens = slower response in conversations

Quality:
Tokenization affects understanding of domain-specific terms

Key insight: Every token costs money, latency and context space.

Sources: Sennrich et al. (2016); T1 – Jurafsky & Martin. Token splits shown for the GPT-4 (cl100k_base) tokenizer.

Tokenization: Cost Impact

Worked Example: Customer-Support Chatbot

List prices per 1M tokens (2025): GPT-4o $2.50 in / $10 out; GPT-4o mini $0.15 in / $0.60 out

- Typical conversation: ~10 turns (user + agent)
- Input: ~5K–10K tokens (system prompt + history re-sent every turn)
- Output: ~1K tokens
- GPT-4o: ~$0.02–0.04 per conversation
- GPT-4o mini: ~$0.001–0.002 per conversation
- 10,000 conversations/day on GPT-4o: ~$225–350/day

Key insight: Model routing and prompt optimization can cut costs 10–20× (Module 4).

Byte-Pair Encoding (BPE): How It Works

• **BPE** is a subword tokenizer between character-level (long sequences, little meaning) and word-level (huge vocabulary, fails on unknown words).

• **How it works**
• **1. Count:** Count adjacent symbol pairs in a large corpus.
• **2. Merge:** Add the most frequent pair (e.g., 'e' + 'r' → 'er') to the vocabulary.
• **3. Repeat:** Iterate to the target vocabulary size; common affixes (un-, -ing) and words emerge.

Training starts from the unique words in the normalized, pre-tokenized corpus.

The base vocabulary is every symbol used to write them.

Example corpus: "hug", "pug", "pun", "bun", "hugs"

Base vocabulary: ["b", "g", "h", "n", "p", "s", "u"]

Sources: Sennrich et al. (2016); Hugging Face LLM Course, Ch. 6.5

BPE Step 1: Count Symbol Pairs

Word frequencies in the corpus:

("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)

For example, "hug" occurs 10 times and "pun" 12 times.

- Split each word into characters (the base vocabulary):

- ("h" "u" "g", 10), ("p" "u" "g", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "u" "g" "s", 5)

- **Count pairs:** ("u", "g") appears in "hug", "pug" and "hugs" → 20 times, the most frequent pair.

Source: Hugging Face LLM Course, Ch. 6.5

BPE Steps 2–3: First and Second Merges

Merge 1: ("u", "g") → "ug" (20 occurrences)

Vocabulary: ["b", "g", "h", "n", "p", "s", "u", "ug"]

Corpus: ("h" "ug", 10), ("p" "ug", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "ug" "s", 5)

Candidates: ("h", "ug") = 15; ("u", "n") = 16

Merge 2: ("u", "n") → "un" (most frequent)

Vocabulary: ["b", "g", "h", "n", "p", "s", "u", "ug", "un"]

Corpus: ("h" "ug", 10), ("p" "ug", 5), ("p" "un", 12), ("b" "un", 4), ("h" "ug" "s", 5)

Source: Hugging Face LLM Course, Ch. 6.5

BPE Step 4: Third Merge & New Words

Merge 3: ("h", "ug") → "hug" (first three-letter token)

Vocabulary: ["b", "g", "h", "n", "p", "s", "u", "ug", "un", "hug"]

Corpus: ("hug", 10), ("p" "ug", 5), ("p" "un", 12), ("b" "un", 4), ("hug" "s", 5)

Repeat until the target vocabulary size is reached.

Learned rules: ("u", "g") → "ug"; ("u", "n") → "un"; ("h", "ug") → "hug"

"bug" → ["b", "ug"]
"mug" → ["[UNK]", "ug"] ("m" not in base vocabulary)
"thug" → ["[UNK]", "hug"] ("t" unknown; rules apply in order)

Note: Byte-level BPE (used by GPT models) never needs [UNK].

Quiz: How will the word "unhug" be tokenized?

Source: Hugging Face LLM Course, Ch. 6.5

Hands-On: Tokenization with tiktoken

Understanding Tokenization with Code

Convert text to tokens with OpenAI's tiktoken library.

Demo Components:

- **Text to tokens:** See how different texts are split
- **Token counting:** Count tokens in a sample conversation
- **Cost analysis:** Estimate the cost of a conversation
- **Model comparison:** Compare tokenizers across models

Context Windows 2024: Crossing 100K

GPT-4 Turbo 128K tokens (~96K words)	Standard production
Claude 3.5 Sonnet 200K tokens (~150K words)	Extended conversations
Gemini 1.5 Pro 1M tokens (~750K words)	Entire codebases
Emerging Models 2M+ tokens (~1.5M words)	Specialized use cases

Bar length: log scale, same scale on all three slides

Where it started: GPT-3.5 (2022) 4K → GPT-4 (2023) 8K/32K → Claude 2.1 (late 2023) 200K.

Challenge: "Lost in the middle": models miss information placed mid-context (Liu et al., 2023). Solution: RAG + memory systems.

Liu et al. (2023), Lost in the Middle; vendor model documentation (2024)

Context Windows 2025: The 1M Club

GPT-5 400K tokens (~300K words)	Agentic, multi-step workflows
Claude Sonnet 4 1M tokens, beta (~750K words)	Large codebases, long agent runs
GPT-4.1 · Gemini 2.5 Pro 1M tokens (~750K words)	Full repos & document sets
Llama 4 Scout (open weights) 10M tokens (~7.5M words)	Largest advertised window

Bar length: log scale, same scale on all three slides

Shift: 1M tokens went from one vendor's feature (2024) to an industry norm; Claude's default stayed at 200K, with 1M in beta.

Challenge: Advertised ≠ effective: accuracy drops well before the limit (RULER, 2024). Solution: context engineering – RAG, compaction, prompt caching.

Hsieh et al. (2024), RULER; OpenAI, Anthropic, Google, Meta model documentation (2025)

Context Windows 2026: 1M Is Standard

Small & efficient models 128K–256K tokens (~96K–190K words)	High-volume, low-cost tasks
Claude Opus 5.5 · Sonnet 5 1M tokens (~750K words)	Whole codebases, long agent traces
GPT-6.1 · Gemini 3.1 Pro ~1M–1.05M tokens (~790K words)	Document sets, video, multimodal
Llama 4 Scout (2025) 10M tokens (~7.5M words)	Still the largest advertised

Bar length: log scale, same scale on all three slides

Plateau: Flagships cluster around 1M; the race has moved from "how big" to "how well is it used", and at what cost and latency.

Direction: Large windows + RAG + persistent memory + context engineering (what to load, summarize, or drop) work together, not as substitutes.

Vendor model documentation and public leaderboards (Oct 2026); verify current figures

Open Problems: Where Research Begins

Consistent multi-step reasoning

LLMs still fail on novel logic outside their training data; Chain-of-Thought helps but does not solve it.

Active research: process reward models, tree-of-thought

Persistent cross-session memory

Each conversation starts blank; external memory is lossy, costly and fragile.

Active research: MemGPT, Titans (Google, 2024)

Grounded factual accuracy

Models state plausible falsehoods; RAG reduces but does not eliminate them.

Active research: attribution, RLHF, verification agents

Reliable long-horizon agent execution

100-step tasks still drift, stall or compound errors.

Active research: agent benchmarks (GAIA, SWE-bench)

Safety & alignment at scale

More autonomy demands agents that follow human intent without side effects.

Active research: Constitutional AI, RLAIF, interpretability

Compute & energy efficiency

Frontier models need huge infrastructure; efficient inference is still open.

Active research: Mamba, QLoRA, mixture-of-experts

Key Takeaways

1 Chatbots

Chatbots answer questions.

2 LLM-Powered Assistants

LLM-powered assistants assist users with tasks.

3 AI Agents

AI agents autonomously manage workflows and pursue goals.

4 Success Depends On

Data grounding (RAG and fine-tuning) and autonomy (reasoning, planning, and orchestration).

5 Domain-Specific Solutions

SAP Joule, Salesforce Einstein, Microsoft Dynamics 365 agents, ServiceNow AI Agents, and Amazon Q Business.

6 Business Value

These drive intelligent automation tailored to specific business needs.

References

Textbook (T1) & Reference (R1)

- T1: Jurafsky, D. & Martin, J. H. Speech and Language Processing (3rd ed. draft), Ch. 1. web.stanford.edu/~jurafsky/slp3
- R1: Wang, L. et al. (2024). A Survey on Large Language Model based Autonomous Agents. Frontiers of Computer Science, 18(6), 186345.

LLMs, Agents & Retrieval

- Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS.
- Brown, T. et al. (2020). Language Models are Few-Shot Learners. NeurIPS.
- Yao, S. et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR.
- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.
- Bommasani, R. et al. (2021). On the Opportunities and Risks of Foundation Models. arXiv:2108.07258.

Tokenization & Context

- Sennrich, R. et al. (2016). Neural Machine Translation of Rare Words with Subword Units. ACL.
- Liu, N. F. et al. (2024). Lost in the Middle: How Language Models Use Long Contexts. TACL.
- Hugging Face LLM Course, Ch. 6.5: Byte-Pair Encoding Tokenization. huggingface.co/learn/llm-course

History, Protocols & Industry Data

- Weizenbaum, J. (1966). ELIZA: A Computer Program for the Study of Natural Language Communication. CACM, 9(1).
- Anthropic (2024). Model Context Protocol. modelcontextprotocol.io · Google (2025). Agent2Agent (A2A) Protocol.
- Grand View Research (2025). Conversational AI Market Report. · Microsoft (2025). Copilot Studio adoption data.